Foundations of Artificial Intelligence

SMART TRAFFIC NAVIGATION SYSTEM

A PROJECT REPORT

CS23533 - Foundations of Artificial Intelligence

Submitted by

2116240701005 – ABENANTHAN P

2116240701003 – AAYAM SUBEDI



October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "Smart Traffic Navigation System" is the bonafide work of "Abenanthan P (240701005)" and "Aayam Subedi (240701003)" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	4
1	INTRODUCTION	5
2	LITERATURE REVIEW	7
3	PROPOSED SYSTEM	9
	3.1 ARCHITECTURE DIAGRAM	10
	3.2 ROAD NETWORK AND DATA SOURCE	11
4	MODULES DESCRIPTION	12
	4.1 MODULE 1: INPUT PROCESSING	12
	4.2 MODULE 2: ROAD NETWORK CREATION	13
	4.3 MODULE 3: WEIGHTED GRAPH MANAGEMENT	13
	4.4 MODULE 4: A* SEARCH	14
	4.5 MODULE 5: ROAD BLOCKING	15
	4.6 MODULE 6: GRAPHICAL UI	16
	4.7 MODULE 7: TESTING AND VALIDATION	17
5	IMPLEMENTATION AND RESULTS	19
	5.1 EXPERIMENTAL SETUP	19
	5.2 RESULTS	20
6	CONCLUSION AND FUTURE WORK	27
7	REFERENCES	29

ABSTRACT

The Smart Traffic Navigation System is designed to find the best route between two locations using the A* search algorithm. The road network is represented as a graph, where locations are nodes and roads are edges with travel costs. The algorithm uses the cost of the current route and an estimated cost to the destination to select a suitable path. The system displays the selected route, total travel cost, and the locations explored during the search. Users can enter their own road network, choose a starting point and destination, and block roads when required. When a road is blocked, the system updates the road network and finds an alternative route if one is available. The system was tested using different cases, including invalid locations, disconnected routes, and road blocking. The results show that the A* algorithm can find suitable routes based on the available road costs. The system also provides a visual representation of the road network to make route selection easier to understand. This project demonstrates how artificial intelligence can be used to solve route-finding problems and support basic traffic navigation.

1. INTRODUCTION

Traffic congestion, roadblocks, and increasing travel time are common problems in transportation. Choosing the right route is important to reach a destination efficiently. Traditional route selection may not always consider the cost of different roads or changes in road conditions. Artificial intelligence can help solve this problem by using search algorithms to find suitable routes based on the available information.

The Smart Traffic Navigation System is developed using the A* search algorithm, an informed search technique used to find a path from a starting location to a destination. The road network is represented as a weighted graph, where locations are nodes and roads are edges with travel costs. A* considers the actual cost from the starting point and the estimated cost to reach the destination. These values are combined to select a suitable route with a lower total cost.

The main aim of this project is to develop a simple navigation system that uses the A* algorithm to find routes between locations. The system displays the selected route, total travel cost, and locations explored during the search process. It also provides a graphical view of the road network, making the route easier to understand. Users can select the starting location and destination or enter their own road network to test different routes.

The system also allows users to block selected roads and find an alternative route based on the updated road network. It uses the A* algorithm to check the available paths and select a suitable route when the original path is no longer accessible. Different test cases are used to check the accuracy of the algorithm, including invalid

locations, disconnected roads, and changes in road availability. The project demonstrates the use of artificial intelligence in route planning and helps users understand how search algorithms can solve navigation problems.

The objectives of this project are:

1. To represent the road network as a weighted graph containing locations and roads.
2. To implement the A* search algorithm to find a suitable route between two locations.
3. To calculate and display the total travel cost of the selected route.
4. To show the search process, including the locations explored by the algorithm.
5. To allow users to enter a custom road network and select the starting point and destination.
6. To provide a road-blocking feature and find an alternative route when a road is unavailable.

The project is implemented using Python, with separate modules for the road network, search algorithm, road management, testing, and graphical user interface. The system is tested with different cases, including valid routes, invalid locations, disconnected networks, and blocked roads. When a road is blocked, the system updates the network and searches for another available route. The project uses a simulated road network rather than live traffic data or real-time GPS information. It demonstrates how artificial intelligence search techniques can be applied to route finding and basic traffic navigation.

2. LITERATURE REVIEW

The Smart Traffic Navigation System uses artificial intelligence search techniques to find suitable routes between locations. Existing navigation systems and route-planning methods help users select paths based on distance, travel cost, and road conditions. The following literature survey reviews related applications and algorithms, their advantages, and their limitations.

Paper/website/App Reference	Problem Description	Tools/ Algorithms used	Advantages	Drawbacks
Google Maps Traffic Ref: https://maps.google.com	Provides real-time navigation by estimating traffic conditions and suggesting optimal routes using live traffic data, historical traffic patterns, and GPS information.	1. A* Search 2. GPS Data Analysis 3.Machine Learning	1. Real-time traffic updates .2. Accurate travel-time estimation. 3. Dynamic rerouting.	1. Depends on internet connectivity. 2. Requires continuous location sharing.
OpenStreetMap Ref: https://www.openstreetmap.org/	Open-source routing engine that computes shortest and fastest routes for road networks using map data from OpenStreetMap.	1. A* Search 2. Routing Optimization	1. Open source 2. Fast route computation. 3. Supports multiple transport modes.	Limited real-time traffic support in the open-source version.

HERE Technologies Routing API Ref: https://www.here.com	Provides intelligent route planning with live traffic information, ETA prediction, and alternate route suggestions for navigation applications.	1. Dijkstra's Algorithm 2.Metaheuristic 3. Local Search	1. High routing accuracy. 2. Real-time traffic integration. 3. Suitable for enterprise applications.	1. Usage limits on free plans. 2. Commercial licensing for large-scale deployment.
A Search Algorithm for Pathfinding Ref: Hart, Nilsson & Raphael (1968),	Uses both actual path cost and heuristic estimates to efficiently determine the optimal route in navigation problems..	1. A* Search 2. Heuristic Function	1. Faster than UCS. 2. Produces optimal routes with admissible heuristics. 3. Widely used in navigation systems.	1. Performance depends on the heuristic quality. 2. More complex than uninformed search algorithms.
Waze Navigation System Ref: https://www.waze.com	Community-driven navigation application that provides real-time traffic information and recommends optimal routes based on live traffic conditions, accidents, road closures, and user reports.	1. GPS Tracking 2.Crowdsource Traffic Data 3. Dynamic Route Optimization 4. Traffic Prediction	1. Real-time traffic updates. 2. Fast rerouting during congestion. 3. Community-based incident reporting.	1. Requires active internet connection. 2. Depends on a large user base for accurate traffic information. 3. Proprietary routing algorithms.

3. PROPOSED SYSTEM

The proposed Smart Traffic Navigation System is developed using Python and the A* search algorithm to find suitable routes between locations. It represents the road network as a weighted graph and calculates the route using the actual cost travelled and the estimated cost to the destination. The system is organised into the following functional parts:

1. **Road Network Creation** – The road network is represented as a graph, where locations are nodes and roads are edges with travel costs.
2. ***A* Search Algorithm*** – The algorithm explores available routes using the actual cost and estimated remaining cost to find a suitable path.
3. **Route Cost Calculation** – The system calculates and displays the total cost of the selected route.
4. **User Input** – Users can select the starting location and destination or enter their own road network for route finding.
5. **Road Blocking** – Users can block a selected road. The system updates the road network and searches for an alternative route if one is available.
6. **Graphical User Interface** – A graphical interface displays the road network, selected route, total cost, explored locations, and road status.
7. **Testing and Validation** - Different test cases are used to check route finding, invalid locations, disconnected roads, and road-blocking operations.

The system calculates routes using available road costs and updates the network when a road is blocked to find an alternative route. It uses a simulated road network without live traffic data or GPS.

3.1. ARCHITECTURE DIAGRAM

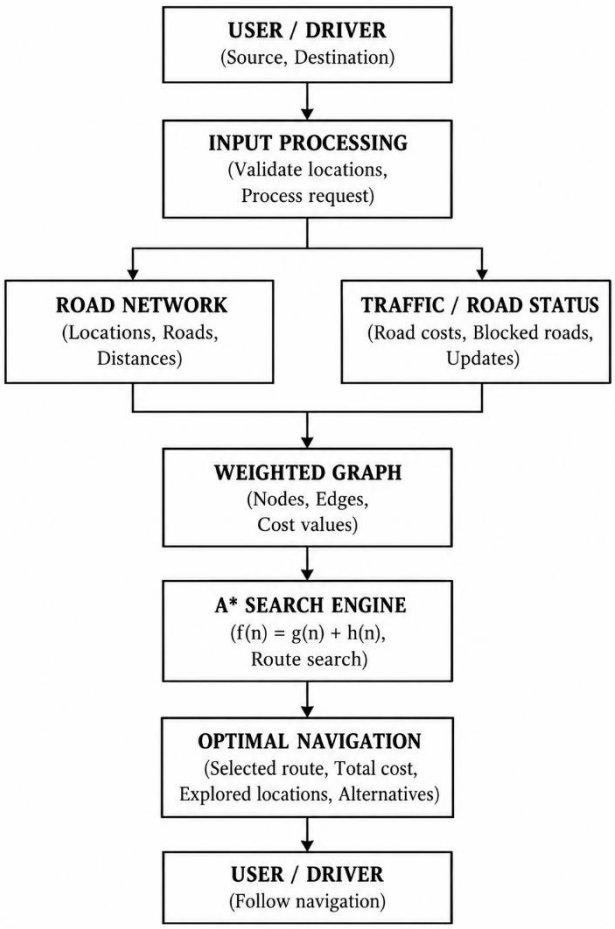


Fig 1: Architecture diagram of the Smart Navigation System

As shown in Fig. 1, the road network and road status are used to build the weighted graph. The A* search engine is the main component of the system. It calculates the route using the actual cost and estimated cost to the destination, then sends the selected route, total cost, and explored locations to the navigation display. When a road is blocked, the system updates the graph and searches for an alternative route.

3.2. ROAD NETWORK AND DATA SOURCE

The Smart Traffic Navigation System uses a road network represented as a weighted graph. The graph contains locations as nodes and roads as edges, with each road assigned a travel cost. The system uses this information to find a suitable route between the selected starting point and destination (Fig. 2).

The road network can be created using the predefined locations and roads or by allowing users to enter their own locations, connections, and road costs. This makes it possible to test the A* algorithm with different network structures and travel costs.

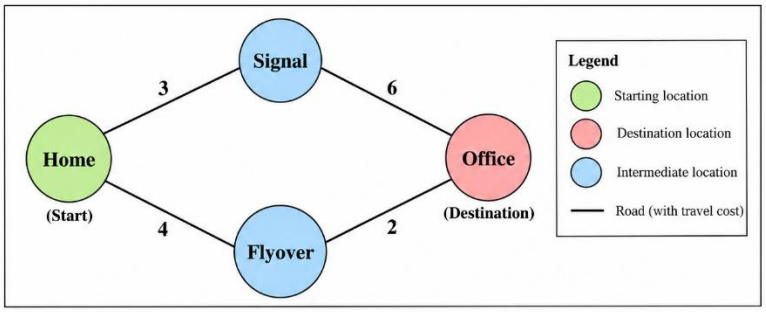


Fig. 2: Weighted Road Network Used by the Smart Traffic Navigation System

Table 1: Main inputs and data used by the system.

Data Item	Description
Locations	Nodes representing places in the road network (Home, Signal, Flyover, Office).
Roads	Connections between locations.
Road Costs	Values assigned to roads for route calculation (e.g., Home–Signal: 3, Home–Flyover: 4, Signal–Office: 6, Flyover–Office: 2).
Source and Destination	Locations selected by the user (e.g., Home to Office).
Blocked Roads	Roads removed from the active network when blocked by the user.
Route Output	Selected path, total travel cost, and explored locations.

Fig. 2: Weighted Road Network for the Smart Traffic Navigation System

The system also supports road blocking. When a user blocks a road, that connection is removed from the active road network. The system then searches for an alternative route using the updated graph. If no valid route is available, it reports that a path could not be found.

4. MODULES DESCRIPTION

The Smart Traffic Navigation System is divided into different modules, each performing a specific task. These modules work together to accept user input, create the road network, find a suitable route using the A* search algorithm, and display the result. The module descriptions are based on the project implementation and the attached module description document.

4.1 MODULE 1: INPUT PROCESSING

This module handles the information provided by the user before the route search begins. The user selects a starting location and a destination from the available locations or enters a custom road network using the input feature.

The module checks whether the selected locations exist in the road network. The starting location represents the point from which the journey begins, while the destination represents the point the user wants to reach. These values are passed to the A* search algorithm along with the road network and heuristic values.

If the user provides an invalid location, the system returns an appropriate message instead of attempting to calculate a route. This helps prevent errors during the search process.

Input: Starting location, destination, and road network.

Output: A validated route request containing the source and destination locations.

4.2 MODULE 2: ROAD NETWORK CREATION

This module creates and manages the road network used by the navigation system. The network is represented as a graph, where each location is a node and each road connecting two locations is an edge. Each road is assigned a cost that represents its value for route calculation.

The system supports a predefined road network as well as a custom network entered by the user. In the custom input feature, the user can provide the number of locations, their names, the number of roads, the connected locations, and the cost of each road. The program then creates the graph from the supplied information.

Road connections are stored so that the search algorithm can identify the available paths between locations. This graph representation makes it easier to compare different routes and calculate their total costs.

Input: Location names, road connections, and road costs.

Output: A graph containing the locations, road connections, and associated costs.

4.3 MODULE 3: WEIGHTED GRAPH MANAGEMENT

The Weighted Graph module prepares the road network for the A* search process. It stores locations as nodes and road connections as edges, with each edge assigned a cost. These costs help the algorithm compare different paths and identify a suitable route from the starting location to the destination.

When the search begins, the algorithm uses the graph to identify the neighbours connected to the current location. It calculates the cost of reaching each neighbour and compares the available options. The graph therefore provides the basic information needed for route calculation.

The system also updates the active graph when a road is blocked. A blocked connection is removed from the available road network, preventing the algorithm from selecting that road during the next search.

Input: Road connections, road costs, and road availability.

Output: An updated weighted graph containing the roads available for route finding.

4.4 MODULE 4: A* SEARCH ALGORITHM

The A* Search module is the main component of the Smart Traffic Navigation System. It finds a route by considering both the actual cost of reaching a location and the estimated cost of travelling from that location to the destination. These values are represented by g(n) and h(n).

The algorithm calculates the estimated total cost using the formula:

$$\mathbf{f(n) = g(n) + h(n)}$$

Here, g(n) represents the actual cost from the starting location to the current location, h(n) represents the estimated remaining cost to the destination, and f(n) represents

their sum. The algorithm selects the location with the lowest estimated total cost for further exploration.

During the search, the algorithm examines the neighbouring locations and calculates their costs. If a better path to a location is found, the cost and path information are updated. This process continues until the destination is reached or no available path remains. The algorithm also records the locations explored during the search, allowing the search process to be displayed.

Input: Weighted graph, starting location, destination, and heuristic values.

Output: Selected route, total route cost, and explored locations, or a message indicating that no route was found.

4.5 MODULE 5: ROAD BLOCKING AND ROUTE RECALCULATION

This module allows users to test how the navigation system responds when a road becomes unavailable. Through the graphical interface, the user can select a road and block it. The selected connection is removed from the active road network, so the A* algorithm cannot use it while calculating the next route.

After blocking a road, the user can run the search again. The algorithm examines the updated graph and attempts to find another available path between the starting location and the destination. If an alternative path exists, the system displays the new route and its total cost. If no valid path is available, it reports that a route cannot be found.

The system also provides a reset option to restore the original road network. This allows users to compare routes before and after blocking a road without permanently changing the predefined network.

Input: Selected road, current road network, starting location, and destination.

Output: Updated road network and a recalculated route, if one is available.

4.6 MODULE 6: GRAPHICAL USER INTERFACE

The Graphical User Interface provides a visual way for users to interact with the Smart Traffic Navigation System. It allows users to select the starting location and destination, find a route, block a road, and reset the road network. The interface also displays the road connections and the status of each road.

When the user requests a route, the interface calls the A* search algorithm and displays the results. The selected route is highlighted on the graph, making it easier to distinguish the chosen path from other connections. The total route cost and the locations explored during the search are also displayed.

Blocked roads are shown differently from available roads, allowing users to identify unavailable connections. After a road is blocked or the network is reset, the user can run the search again to view the updated route.

Input: User selections and route-search requests.

Output: A graphical road network showing the selected route, total cost, explored locations, and road status.

4.7 MODULE 7: TESTING AND VALIDATION

The Testing and Validation module checks whether the route-finding algorithm and its supporting features work as expected. Different test cases are used to verify the results of the A* search algorithm under normal and exceptional conditions.

The tests include finding the expected route in the sample road network, checking cases where the starting location and destination are the same, handling invalid locations, and identifying situations where no path connects the source and destination. Additional tests check whether road costs and connections are handled correctly and whether blocking a road changes the available route.

The results are compared with the expected outcomes to identify errors in route calculation and road network management. These tests help verify that the algorithm returns a suitable route when one is available and handles invalid or disconnected cases appropriately.

Input: Test graphs, source and destination locations, road costs, and road-blocking cases.

Output: Test results indicating whether the route-finding algorithm and related functions behave as expected.

4.8 MODULE WORKFLOW

The modules work together to complete the route-finding process. First, the user selects a starting location and destination. The system prepares the weighted graph and passes the route request to the A* search algorithm. The algorithm explores the available locations and calculates the total cost of the selected route. The graphical interface then displays the route and its details. If a road is blocked, the graph is updated and the search can be performed again to find an alternative path.

5. IMPLEMENTATION AND RESULTS

5.1 EXPERIMENTAL SETUP

The Smart Traffic Navigation System was implemented in Python using the A* search algorithm. The system was developed to find suitable routes between locations using a weighted graph. The experimental setup consists of the following components:

Programming Language: Python was used to implement the A* algorithm, road network management, user input, and graphical interface.

Libraries and Modules: The heapq module is used to manage the priority queue during the A* search. Tkinter is used to create the graphical user interface. Other Python modules are used to manage the road network, road blocking, and testing.

Data: The system uses a predefined road network or a custom network entered by the user. The network contains locations, road connections, and their costs.

Execution: The program is started by running the main Python file. The system loads the road network and allows the user to select a starting location and destination through the interface.

Input: The inputs include the starting location, destination, road connections, road costs, and selected roads to be blocked.

Output: The system displays the selected route, total route cost, explored locations, and road status. If no valid route is available, it displays an appropriate message.

Test Method: Different test cases are used to check route calculation, invalid locations, disconnected networks, and road-blocking operations. The results are compared with the expected outcomes to verify the system's behaviour.

5.2 RESULTS

The Smart Traffic Navigation System was tested using different road networks and input conditions to check the performance of the A* search algorithm. The tests focused on route calculation, total cost, invalid locations, disconnected networks, and road blocking. The results were compared with the expected output to verify whether the system worked correctly.

Table 5: Results of the functional tests.

No.	Test Case	Expected Result	Result
1	Find a route between Home and Office	The system finds Home → Flyover → Office with a total cost of 6.	Passed
2	Select the same starting point and destination	The system returns a route with a cost of 0.	Passed
3	Enter an invalid starting location	The system displays an appropriate error message.	Passed
4	Select locations with no connecting path	The system reports that no route is available	Passed
5	Block a road on the selected route	The system updates the road network and searches for an alternative route.	Passed
6	Reset blocked roads	The system restores the original road network.	Passed
7	View the route on the graphical interface	The selected route and road network are displayed.	Passed

The test cases check the main functions of the system, including route finding, cost calculation, invalid input handling, and road availability. The sample road network produces the route Home → Flyover → Office with a total cost of 6. When a road is blocked, the system can search the updated network for another available route.

These tests help confirm the behaviour of the A* algorithm and the graphical interface under different conditions.

Worked example: For the Smart Traffic Navigation System, consider a route request from Home to Office using the sample road network. The A* algorithm evaluates the available routes using the actual cost g(n) and estimated cost h(n). It calculates the total estimated cost using f(n) = g(n) + h(n) and explores locations based on these values. The available routes are Home → Signal → Office, with a total cost of 9, and Home → Flyover → Office, with a total cost of 6. The algorithm selects Home → Flyover → Office as the optimal route, with a total cost of 6. This example shows how A* compares route costs and uses heuristic values to guide the search towards the destination.

```
Smart Traffic Navigation System
Available Locations: ['Home', 'Signal', 'Flyover', 'Office']
Enter Start Location: Home
Enter Destination: Office

Step 1
Selected Node: Home
Actual Cost g(n): 0
Heuristic h(n): 5
Total Cost f(n): 5
Current Path: Home
Visited Nodes: {'Home'}
Adding: Signal
  g(n) = 3
  h(n) = 5
  f(n) = 8
Adding: Flyover
  g(n) = 4
  h(n) = 2
  f(n) = 6

Step 2
Selected Node: Flyover
Actual Cost g(n): 4
Heuristic h(n): 2
Total Cost f(n): 6
```

```
Current Path: Home -> Flyover
Visited Nodes: {'Home', 'Flyover'}
Adding: Office
  g(n) = 6
  h(n) = 0
  f(n) = 6

Step 3
Selected Node: Office
Actual Cost g(n): 6
Heuristic h(n): 0
Total Cost f(n): 6
Current Path: Home -> Flyover -> Office
Goal Found!
```

Graphical interface. The graphical interface displays the road network, selected route, total travel cost, explored locations, and road status. Users can select the starting location and destination, find a route, block a road, and reset the road network. When a road is blocked, the updated network can be used to find an alternative route. The interface presents the results visually, making it easier to understand how the A* algorithm searches for a route. Figures 5(a) – 5(c) show the interface and its route-finding results under different conditions.

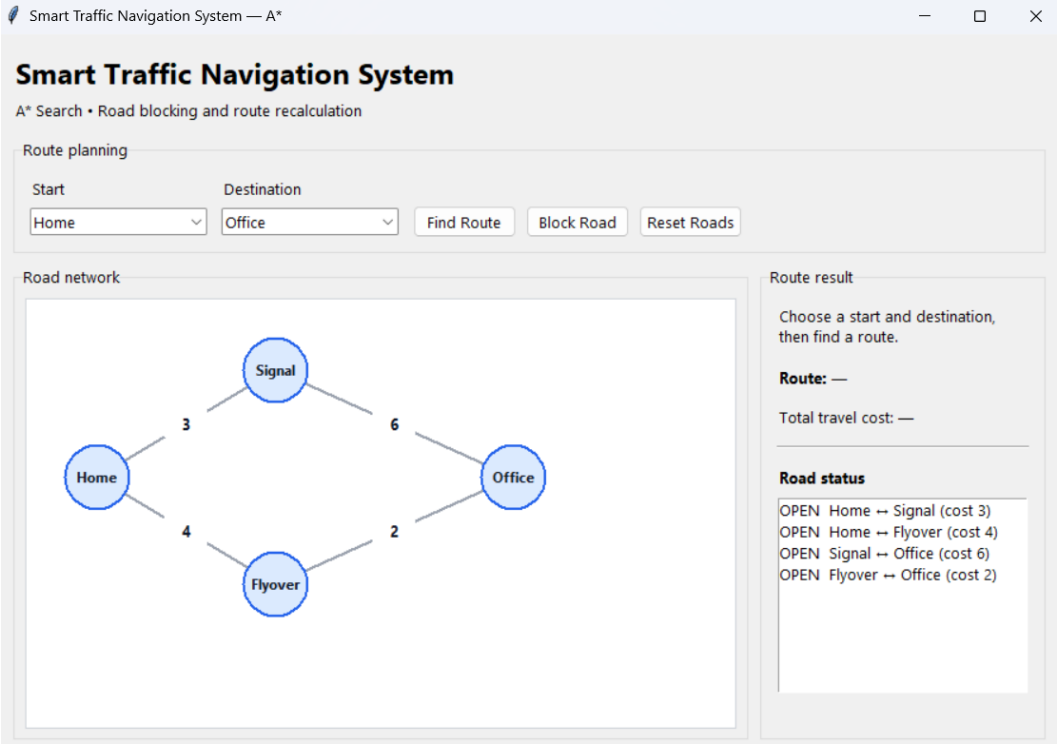


Fig 5: (a) Graphical User Interface of the Smart Traffic Navigation System

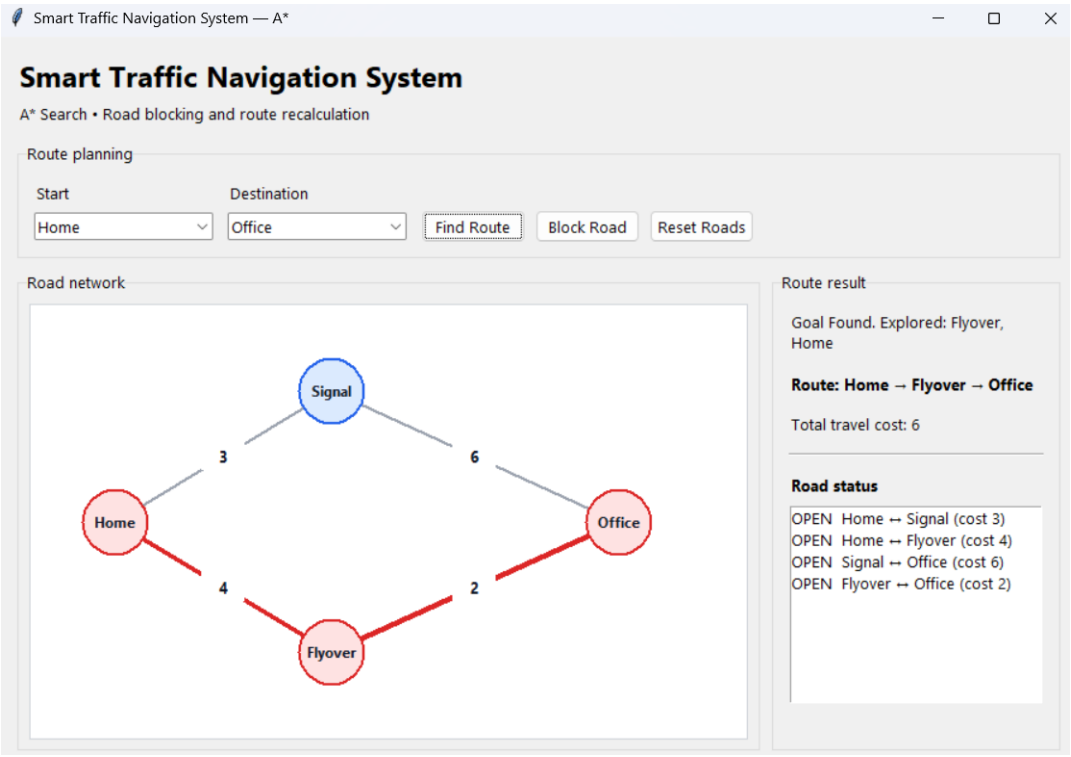


Fig 5: (b) Optimal Route from Home to Office Using the A* Search Algorithm

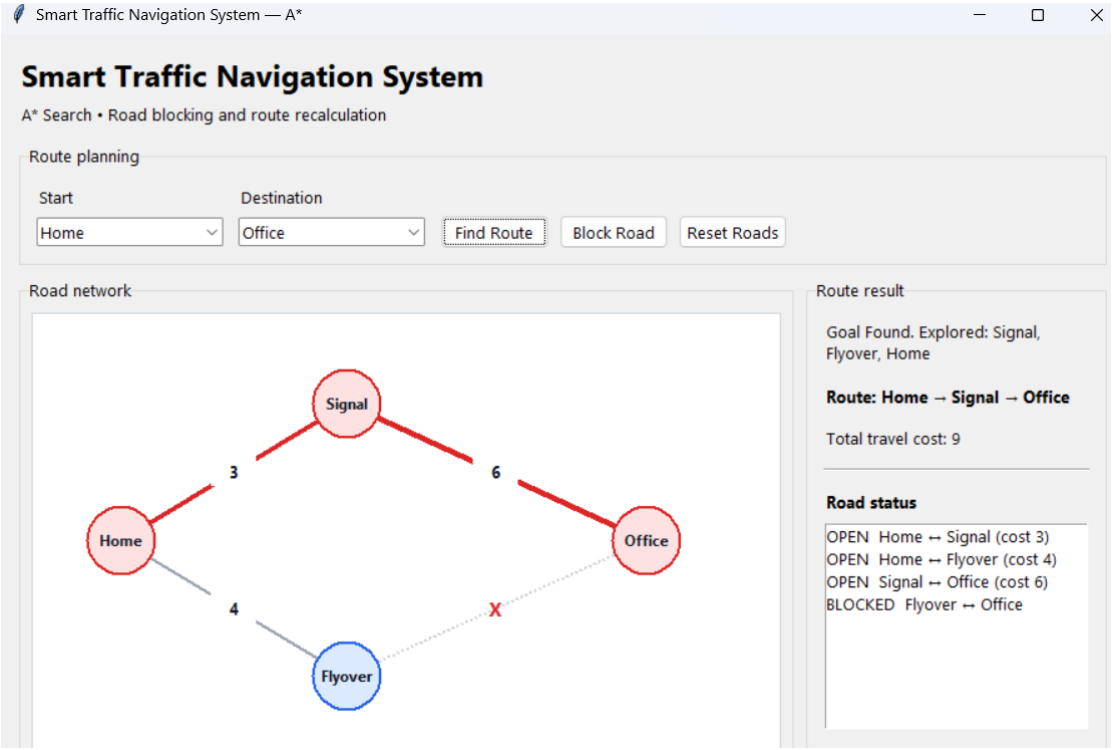


Fig 5: (c) Alternative Route After Blocking a Road

Edge cases: A few additional inputs were tested to examine how the Smart Traffic Navigation System handles unusual conditions and invalid route requests. The results are summarized in Table 6.

Table 6: Behaviour of the Smart Traffic Navigation System on edge cases.

Test Case	Observed behaviour	Explanation
Start and destination are the same	The system returns a route with a total cost of 0.	No travel is required because the starting location is already the destination.
Invalid starting location	The system reports an appropriate error or invalid input status.	The starting location does not exist in the road network.
No available path between locations	The system reports that no route is available.	The road network does not contain a valid connection between the selected locations.

Test Case	Observed behaviour	Explanation
A road on the selected route is blocked	The system recalculates the route using the updated road network.	The blocked road is unavailable, so the algorithm searches for an alternative path.
All possible routes are blocked	The system reports that no valid route can be found.	No available connection leads from the starting location to the destination.

Observations:

The results show that the Smart Traffic Navigation System has several important strengths. The A* search algorithm considers both the actual cost from the starting location and the estimated cost to the destination when selecting a route. In the sample road network, the system selects Home → Flyover → Office, with a total cost of 6, instead of Home → Signal → Office, which has a cost of 9.

The system also supports changes in road availability. When a road is blocked, the updated road network can be used to search for an alternative route. This demonstrates how the system responds to simulated road closures without requiring changes to the entire graph.

The graphical user interface makes the route-finding process easier to understand. It displays the road network, highlights the selected route, shows the total travel cost, lists the explored locations, and indicates whether roads are open or blocked. Users can select different starting points and destinations to test the algorithm under different conditions.

However, the system has some limitations. It uses a simulated road network rather than live geographical data and does not collect real-time traffic conditions or GPS locations. Road costs are predefined or supplied by the user, so the calculated route depends on the accuracy of these values. The quality of the route selected by A* also depends on the heuristic values used to estimate the remaining cost. Furthermore, the current implementation is intended to demonstrate route-finding concepts on a limited road network and has not been established as a large-scale navigation solution. Road closures must also be handled through user interaction rather than being detected automatically.

Overall, the Smart Traffic Navigation System demonstrates the practical application of the A* search algorithm for route planning, travel cost calculation, and alternative route selection within a simulated road network.

6. CONCLUSION AND FUTURE WORK

Conclusion: The Smart Traffic Navigation System was developed to find suitable routes between locations using the A* search algorithm. The road network is represented as a weighted graph, where locations are nodes and roads are edges with travel costs. The algorithm evaluates the actual cost from the starting point and the estimated cost to the destination to select a route with a lower total cost. The system displays the selected route, total travel cost, and explored locations through a graphical user interface.

The project also supports custom road networks and road blocking. When a road is blocked, the system updates the available connections and searches for an alternative route if one exists. Testing covers different situations, including valid routes, identical starting and destination locations, invalid locations, disconnected paths, and changes in road availability. The graphical interface makes it easier to understand the route-finding process and observe the behaviour of the A* algorithm.

The project demonstrates how artificial intelligence search techniques can be applied to route planning and cost-based decision-making. Although the current implementation uses a simulated road network rather than live traffic data, it provides a foundation for understanding intelligent navigation and alternative route selection.

Future work: The current implementation can be extended in several ways:

- **Real-Time Traffic Information:** Integrating live traffic data to update road costs according to congestion and travel conditions.

- **GPS Integration:** Using the user's current geographical location as the starting point for navigation.
- **Map Integration:** Incorporating real geographical maps to represent roads and locations more accurately.
- **Dynamic Road Updates:** Automatically detecting road closures and updating the network without requiring manual input.
- **Improved Heuristic Calculation:** Developing suitable heuristic functions to improve route-search efficiency while maintaining optimality under the required conditions.
- **Larger Road Networks:** Evaluating the system with more locations and road connections to study its performance on larger graphs.
- **Enhanced User Interface:** Adding clearer route instructions, distance information, and more detailed road-status indicators.

These improvements would extend the existing system while retaining the A* search algorithm as the core route-finding method. They could help develop the current simulation into a more practical navigation application.

7. REFERENCES

1. **Russell, S. and Norvig, P. (2020). *Artificial Intelligence: A Modern Approach*. 4th ed. Hoboken, NJ: Pearson.**
   Reference for artificial intelligence search techniques, heuristic search, and the A* search algorithm.
2. **Hart, P. E., Nilsson, N. J. and Raphael, B. (1968). 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths'. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.**
   Foundational research paper introducing the A* search algorithm for minimum-cost pathfinding.
3. **Python Software Foundation (n.d.). *heapq — Heap queue algorithm*.**
   Used for managing the priority queue during the A* search process.
   Python heapq Documentation
4. **Python Software Foundation (n.d.). *tkinter — Python interface to Tcl/Tk*.**
   Used to develop the graphical user interface for route selection and road network visualization.
   Python tkinter Documentation
5. **Python Software Foundation (n.d.). *unittest — Unit testing framework*.**
   Reference for implementing and executing test cases to verify the system's behaviour.
   Python unittest Documentation
6. **Cormen, T. H., Leiserson, C. E., Rivest, R. L. and Stein, C. (2022). *Introduction to Algorithms*. 4th ed. Cambridge, MA: MIT Press.**
   Reference for graph representations, priority queues, and graph-based algorithm design.